Principles of Feedback Control Project

Wesley Hon
EC ENGR 141
Professor Xiaofan Cui

March 15th, 2026

Abstract

This project analyzes and controls a DC power system supplying a constant power load (CPL) through an L-R-C network. The system is first modeled using nonlinear state-space equations with the inductor current and capacitor voltage selected as the state variables. The nonlinear dynamics arise from the constant power load, which introduces a nonlinear current term proportional to $\frac{P}{v_{DC}}$. The model is then linearized around an equilibrium operating point to obtain a linear state-space representation. From this linearized model, the transfer function relating the input voltage perturbation to the output capacitor is derived and the stability of the system is analyzed.

Using the obtained linear model, a controller is then designed and implemented in Simulink to meet specified time-domain requirements, including a settling time of less than 0.05s and no overshoot. The controller is first tested on the linearized system and then applied to the original nonlinear system while enforcing physical safety constraints through saturation limits on voltage and current. Simulation results demonstrate the behavior of the controlled system and show differences between the linearized and nonlinear models. The results illustrate the destabilizing effects of constant power loads and the importance of appropriate controller design for maintaining stability in DC power systems.

Contents

1 State-Space Representation of the System 4

2 Linearized Dynamics 5

3 Open Loop Transfer Function 8

4 Equilibrium Values for the Current through the Wire 10

5 Stability 12

6 Controller Design 14

7 Simulink Implementation 21

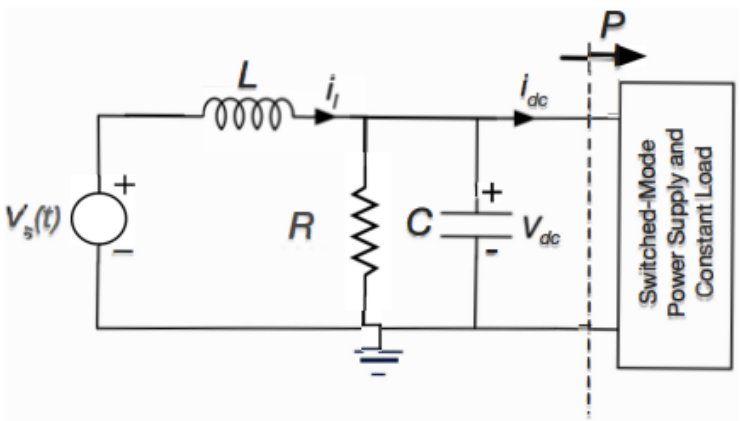

Figure 1: Circuit schematics of the system described above

Defining the Physical System

Circuit Components

$V_s(t)$	(DC source input)
L	(Inductor)
C	(Capacitor)
R	(Parallel Resistor)
Cosntant Power Load (CPL)	modeled as power P

States

Capacitor node voltage: v_{dc}
Inductor current: i_l
$\hookrightarrow$ states of the system.

CPL

CPL draws constant power. No constant resistance.

Power relation: $P = v_{dc} * i_{dc}$

Thus, $i_{dc} = \dfrac{P}{v_{dc}}$

The system is nonlinear.

State Variables

$$x_1(t) = i_l(t)$$
$$x_2(t) = v_{dc}(t)$$

Input: $u(t) = v_s(t)$
Output: $y = v_{dc}$

1 State-Space Representation of the System

Considering the inductor current and the capacitor voltage as your two states (i.e., $x_1(t) = i_1(t)$ and $x_2(t) = v_{dc}(t)$), obtain the state-space representation for this system in the form of $\dot{x} = f(x, u)$ where $u(t) = v_s(t)$. Recall that power supplied to the load is the product of the current through and the voltage across it.

Goal: Solve for the state space representation of the system in the form $\dot{x} = f(x, u)$.

Analyzing the circuit, the circuit components are one inductor (L) in series and two branches of a resistor and a capacitor (C) down to ground.

> $\dot{x_1}$ through the voltage across the inductor
>
> $$L\frac{di_l}{dt} = v_s - v_{dc} \qquad \text{voltage across the inductor}$$
> $$\dot{x_1} = \frac{1}{L}(v_s - v_{dc}) \qquad \text{where } x_1 = i_l$$

> $\dot{x_2}$ through the voltage across the capacitor
>
> $$C\frac{dv_{dc}}{dt} = i_l - i_R - i_{dc} \qquad \text{current through the capacitor}$$
>
> > Resistor current: $i_R = \dfrac{v_{dc}}{R}$
> >
> > CPL current: $i_{dc} = \dfrac{P}{v_{dc}}$
>
> $$C\frac{dv_{dc}}{dt} = i_l - \frac{v_{dc}}{R} - \frac{P}{v_{dc}} \qquad \text{sub the currents}$$
> $$\dot{x_2} = \frac{1}{C}(i_l - \frac{v_{dc}}{R} - \frac{P}{v_{dc}}) \qquad \text{where } x_2 = v_{dc}$$

Nonlinear Model – State Space Representation of the System

$$\boxed{\begin{aligned} \dot{x_1} &= \frac{1}{L}(v_s - x_2) \\ \dot{x_2} &= \frac{1}{C}(x_1 - \frac{x_2}{R} - \frac{P}{x_2}) \end{aligned}}$$

where:
$\hookrightarrow x_1 = i_l(t)$
$\hookrightarrow x_2 = v_{dc}(t)$

```
Command Window
>> syms x1 x2 u R L C P real
>> f1 = (1/L)*(u-x2);
>> f2 = (1/C)*(x1-x2/R-P/x2);
>> f=[f1;f2]

f =

           (u - x2)/L
-(P/x2 - x1 + x2/R)/C
```

Code Snippet 1: This MATLAB script shows the creation of the variables x_1, x_2, u, R, L, C, P. It defines f_1 and f_2 as variables for the state space representation $\dot{x_1}$ and $\dot{x_2}$ respectively. Additionally printing f as the matrix of the state space representation of the system.

2 Linearized Dynamics

Assume that the system has an equilibrium point at $(i_{l_0}, v_{dc_0}, v_{s_0})$. Obtain the linearized dynamics of this system in the form of $\delta\dot{x} = A\delta x + B\delta u$ where $\delta x \triangleq [i_l - i_{l0} \quad v_{dc} - v_{dc0}]^T$ and $\delta u \triangleq v_s - v_{s0}$. Find matrices A and B.

Goal: Get the linearized dynamics of the system in the form of $\delta\dot{x} = A\delta x + B\delta u$ and find the matrices A and B.

Linearize around the equilibrium: $(i_{l0}, v_{dc0}, v_{s0})$

Deviations:

$$\delta x = \begin{bmatrix} i_l - i_{i0} \\ v_{dc} - v_{dc0} \end{bmatrix}$$
$$\delta u = v_s - v_{s0}$$

Linear system form: $\delta \dot{x} = A\delta x + B\delta u$
Deviations from the equilibrium point is given by a deviation in x and a deviation in u that causes a total deviation in the system $\dot{x}$.

Matrix A

Partial derivative of $f(x, u)$

First Equation:

$$\dot{x_1} = \frac{1}{L}(v_s - x_2)$$

Derivatives:

$$\frac{\partial}{\partial x_1} = 0$$
$$\frac{\partial}{\partial x_2} = -\frac{1}{L}$$

Second Equation:

$$\dot{x_2} = \frac{1}{C}(x_1 - \frac{x_2}{R} - \frac{P}{x_2})$$

Derivatives:

$$\frac{\partial}{\partial x_1} = \frac{1}{C}$$
$$\frac{\partial}{\partial x_2} = \frac{1}{C}(-\frac{1}{R} + \frac{P}{v_{dc0}^2})$$

Thus,

$$\boxed{A = \begin{bmatrix} 0 & -\frac{1}{L} \\ \frac{1}{C} & \frac{1}{C}(-\frac{1}{R} + \frac{P}{v_{dc0}^2}) \end{bmatrix}}$$

Matrix B

Only the first equation depends on v_s (the input).
Derivative with respect to input v_s: $\frac{1}{L}$.

$$\boxed{B = \begin{bmatrix} \frac{1}{L} \\ 0 \end{bmatrix}}$$

As the Jacobian of f with respect to x and u make up the matrices **A** and **B**

$$A = \left.\frac{\partial f}{\partial x}\right|_{(x_0,u_0)}, \quad B = \left.\frac{\partial f}{\partial u}\right|_{(x_0,u_0)}$$

ece141_project_q2.m

C:\Users\wesle\OneDrive\Documents\MATLAB\ece141_project_q2.m

```
syms x1 x2 u R L C P real

f1 = (1/L)*(u - x2);
f2 = (1/C)*(x1 - x2/R - P/x2);

f = [f1; f2];
x = [x1; x2];

A_sym = jacobian(f, x);
B_sym = jacobian(f, u);

disp('A = ')
pretty(A_sym)

disp('B = ')
pretty(B_sym)
```

Command Window

```
>> ece141_project_q2
A =
/        1   \
| 0,   - -   |
|        L   |
|            |
|     P    1 |
|    --- - - |
|      2   R |
| 1  x2      |
| -, ------- |
\ C     C    /

B =
/ 1 \
| - |
| L |
|   |
\ 0 /

>>
```

Code Snippet 2:
This MATLAB script shows the definition of the variables x_1, x_2, u, R, L, C, P, as seen in Part 1.

We then construct the matrices f and x. f is the state space representation of the system and x is defined to be the state space variables.

The matrices A and B are then defined as the Jacobian of f with respect to x and u, respectively.

> The matrix A is defined to be the State Matrix that defines the changes to the internal state dynamics.
>
> The matrix B is defined to be the Input Matrix that defines how inputs affect states.

The matrices are then displayed on Matlab script.
The displayed matrices match our calculated matrix values.

3 Open Loop Transfer Function

Find the open-loop transfer function $H(s)$ from input δv_s to output δv_{dc} in s-domain, assuming zero initial conditions. Find the relationship between R, L, C, and P that would ensure the linearized system would remain stable.

Transfer Function

$$\begin{aligned} \text{Linear System TF} \qquad & H(s) = \frac{\delta v_{dc}(s)}{\delta v_s(s)} \\ \text{Using:} \qquad & H(s) = C(sI - A)^{-1}B \\ \text{Where:} \qquad & C_{out} = \begin{bmatrix} 0 & 1 \end{bmatrix} \end{aligned}$$

This concludes to be a second order transfer function.

```
syms vdc0 real

A_eq = subs(A_sym, x2, vdc0);
B_eq = B_sym;

R = 1000;
P = 20;
L = 100e-6;
C = 100e-6;
vdc0 = 80;

A = [0, -1/L;
     1/C, (-1/R + P/vdc0^2)/C];

B = [1/L; 0];
```

Code Snippet 3: This MATLAB script shows the initialization of the variables R, P, L, C, and v_{dc0} so that the following transfer function code will compile. Matlab requires real number values.

These variables are fetched from the project spec.

```
Cout = [0 1];
D = 0;

sys_ss = ss(A,B,Cout,D);
H = tf(sys_ss)
```

Command Window

```
H =

          1e08
  -------------------
  s^2 - 21.25 s + 1e08

Continuous-time transfer function.
Model Properties
>>
```

Code Snippet 4: This MATLAB script shows the initialization of the matrices C and D and calculates the transfer function from all four state-space matrices.

The matrix C maps the states to the output. It describes what part of the internal system states you observe.

The matrix D maps the input directly to the output. It describes any instantaneous effects of the input on the output.

Calculated Transfer Function

From the Matlab simulation:

$$\boxed{H(s) = \frac{10^8}{s^2 - 21.25s + 10^8}}$$

Stability

For a 2nd order system, stability requires that:

$\hookrightarrow$ trace$(A) < 0$

$\hookrightarrow$ det$(A) > 0$

Trace

$$
\begin{aligned}
tr(A) &= \frac{1}{C}(-\frac{1}{R} + \frac{P}{v_{dc0}^2}) \\
\frac{1}{C}(-\frac{1}{R} + \frac{P}{v_{dc0}^2}) &< 0 && \text{need to be negative trace} \\
-\frac{1}{R} + \frac{P}{v_{dc0}^2} &< 0 && \text{since } C > 0 \\
\frac{P}{v_{dc0}^2} &< \frac{1}{R} \\
P &< \frac{v_{dc0}^2}{R}
\end{aligned}
$$

Determinant

$$\det(A) = \frac{1}{LC} > 0$$

which is already true, since $L, C > 0$.

$$\boxed{\frac{P}{v_{dc0}^2} < \frac{1}{R}}$$

Constant power loads behave like negative resistance.
Too much power would make the system unstable.

4 Equilibrium Values for the Current through the Wire

Assume the parallel resistance of $R = 1000\Omega$, the CPL power constant of $P = 20W$ and, input equilibrium DC voltage of $v_{s0} = 80V$. Find all possible corresponding equilibrium values for the current through the wire and the voltage across the outdoor converter (i.e., i_{l0} and v_{dc_0} respectively.)

Goal: Find the equilibrium values for the current through the wire and the voltage across the outdoor converter.

Given values: $R = 1000\Omega$, $P = 20W$, $v_{s0} = 80V$.
$\hookrightarrow$ At equilibrium, $\dot{x_1} = 0$, $\dot{x_2} = 0$.

> From inductor equation
>
> $$0 = \frac{1}{L}(v_{s0} - v_{dc0})$$
> $$v_{dc0} = v_{s0} = 80V$$
>
> From capacitor equation
>
> $$0 = i_{l0} - \frac{v_{dc0}}{R} - \frac{P}{v_{dc0}}$$
> $$i_{l0} = \frac{v_{dc0}}{R} + \frac{P}{v_{dc0}}$$
>
> Resistor current: 80/1000 = 0.08A.
> CPL current: 20/80 = 0.25A.
>
> $$i_{l0} = 0.08A + 0.25A = 0.33A$$
>
> Thus,
>
> $$\boxed{i_{l0} = 0.33A}$$

```
R = 1000;
P = 20;
vs0 = 80;

vdc0 = vs0;
iL0 = vdc0/R + P/vdc0;

fprintf('vdc0 = %.4f V\n', vdc0);
fprintf('iL0  = %.4f A\n', iL0);

syms iL0 vdc0 vs0 R P real

eq1 = vs0 - vdc0 == 0;
eq2 = iL0 - vdc0/R - P/vdc0 == 0;

sol = solve([eq1,eq2],[iL0,vdc0]);
sol.iL0
sol.vdc0
```

Command Window

```
vdc0 = 80.0000 V
iL0  = 0.3300 A

ans =

(vs0^2 + P*R)/(R*vs0)


ans =

vs0

>>
```

Code Snippet 5: Using the given values: $R = 1000\Omega, P = 20W, v_{s0} = 80V$, the Matlab script uses the previous inductor and capacitor KVL/KCL equations to solve for the capacitor voltage (v_{dc0}) and the inductor current (i_l).

As expected, the solved values of v_{dc0} and i_l0 match our calculated values: $80V$ and $0.33A$ respectively.

5 Stability

Consider the system linearized around the equilibrium point with thw *lowest* possible value you obtained for v_{dc_0} in Part 4. Assume the same values for R and P as in Part 4 with the addition of $C = 100\mu F$ and $L = 100\mu H$. Is this linearized system BIBO stable? Recall Part 3.

Goal: Checking if the linearized system around the lowest equilibrium point is BIBO stable.

Given: $C = 100\mu F$, $L = 100\mu H$

Condition from Part 3: $\dfrac{P}{v_{dc0}^2} < \dfrac{1}{R}$

Computing this inequality using:
$\hookrightarrow P = 20W$
$\hookrightarrow v_{dc0}^2 = 80V$
$\hookrightarrow R = 1000\Omega$

LHS: $\dfrac{P}{v_{dc0}^2} = \dfrac{20}{80^2} = 0.003125$
RHS: $\dfrac{1}{R} = \dfrac{1}{1000} = 0.001$

Since $0.003125 > 0.001$, the system becomes **unstable**.

> **Using trace:**
> Plugging in the values into the matrix $\mathbf{A}$: $L = 100{*}10^{-6}, C = 100{*}10^{-6}, v_{dc0} = 80$
>
> $$\mathbf{A} = \begin{bmatrix} 0 & -10000 \\ 10000 & \dfrac{-1/1000 + 20/80^2}{100 * 10^{-6}} \end{bmatrix}$$
> $$\mathbf{A} = \begin{bmatrix} 0 & -10000 \\ 10000 & 21.25 \end{bmatrix}$$
>
> $\mathrm{tr}(A) = 0 + 21.25 = 21.25 > 0$
>
> Since $\mathrm{tr}(A) > 0$, the system becomes **unstable**.

Thus, the linearized system is **not BIBO stable**.
The system's poles are not both in the left-half plane. The stability condition defined in Part 3 is violated.

Thus, the *CPL* destabilizes the system.

```
R = 1000;
P = 20;
L = 100e-6;
C = 100e-6;
vdc0 = 80;

A = [0, -1/L;
     1/C, (-1/R + P/vdc0^2)/C];

B = [1/L;0];
Cout = [0 1];
D = 0;

sys = ss(A,B,Cout,D);

eigA = eig(A)

isstable_tf = isstable(tf(sys))
```

Command Window

```
isstable_tf =

  logical

   0
```

Code Snippet 6:
This Matlab script takes in the values for R, P, L, C, and v_{dc0}, redefines the matrices (**A, B, C, D**), defines the eigenvector of **A**, and lastly, solves if the system as a whole is stable.

The results in the command window show that the system is in fact **unstable**.

6 Controller Design

Consider the same linearized system as in Part 5, with all the same values. Let $y = \delta x_2$. Recall the open-loop transfer function H. Build the linearized model in Simulink. Use a solver for stiff differential equations (e.g., ode23s) when using Simulink. Design a controller such that the closed-loop system's output decays to 0 in no more than 0.05s with no overshoot and the initial conditions of $\delta x(0) = [8 \quad -8]$.

Goal: Design a controller such that the output decays to 0 in 0.05 seconds or less. There shall be no overshoot.

Given specs:
$\hookrightarrow$ Settling time $\leq 0.05s$
$\hookrightarrow$ No overshoot
$\hookrightarrow$ Initial condition:

$$\hookrightarrow \delta x(0) = \begin{bmatrix} 8 \\ -8 \end{bmatrix}$$

Letting $y = \delta x_2$

The system is disturbed from equilibrium and the goal is to make the output return to zero quickly and smoothly.

Restating Values

Matrices: $\mathbf{A} = \begin{bmatrix} 0 & -\frac{1}{L} \\ \frac{1}{C} & \frac{1}{C}(-\frac{1}{R} + \frac{P}{v_{dc0}^2}) \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} \frac{1}{L} \\ 0 \end{bmatrix}$

Circuit values: $R = 1000\Omega, P = 20W, L = 100\mu H, C = 100\mu F, v_{dc0} = 80V$

$\frac{1}{L} = 10000$, $\frac{1}{C} = 10000$.
$\frac{1}{C}(-\frac{1}{R} + \frac{P}{v_{dc0}^2}) = \frac{1}{100 * 10^{-6}}(-0.001 + \frac{20}{80^2}) = 21.25$

Therefore, $\mathbf{A} = \begin{bmatrix} 0 & -10000 \\ 10000 & 21.25 \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} 10000 \\ 0 \end{bmatrix}$

The output is stated to be: $y = \delta x_2$.

So, $C_{out} = \begin{bmatrix} 0 & 1 \end{bmatrix}$, $D = 0$

Transfer Function

From the linearized state-space model, the transfer function from δv_s to δv_{dc} is

$$H(s) = \frac{\delta v_{dc}(s)}{\delta v_s(s)}$$

We found that the transfer function becomes:

$$H(s) = \frac{10^8}{s^2 - 21.25s + 10^8}$$

The denominator $(s^2 - 21.25s + 10^8)$ has a negative coefficient, which shows that the system is <u>unstable</u>, since all of the coefficients in the denominator of the transfer function of a system must be positive.

Why a plain proportional controller is not enough

$$C(s) = K_p$$

$\hookrightarrow$ trying a proportional (P) controller $C(s)$.

The closed-loop characteristic equation becomes

$$1 + K_p H(s) = 0$$

$$s^2 - 21.25s + 10^8(1 + K_p)$$

The " $-21.25s$" term still contains a negative coefficient.
Thus, we may not use a proportional controller, since this controller cannot fix the unstable damping term.

In other words, **P** only is not enough. We must use a controller that change the s-coefficient, using derivative action or a lead compensator.

PD Controller

Using a PD controller:

$$C(s) = K_p + K_d s$$

equivalent to PID with $K_i = 0$

Characteristic Polynomial becomes:

$$s^2 + (10^8 K_d - 21.25)s + 10^8(1 + K_p)$$

This can potentially get rid of the negative coefficient in the controller to make it stable. Now, the coefficient $10^8 K_d - 21.25$ must be positive. Since we introduced K_p as well, the final term: $10^8(1 + K_p)$ must be positive as well.

Since we do not want overshoot, the closed-loop poles must be real and negative, perfectly well damped. Choose two real poles p_1, p_2.

$$T_s \approx \frac{4}{|pole|}$$

$$T_s \leq 0.05$$

$\hookrightarrow$ thus, $|pole| \geq 80$.

An option is to use a PD controller to make the system overdamped, so that guarantees no overshoot.

> Assuming $K_p = 0$ since the last term will always be positive, we then calculate the K_d needed to make this system overdamped.
>
> $$\begin{aligned} D &= b^2 - 4ac > 0 \\ D &= (10^8 K_d - 21.25)^2 - 4 * 10^8 > 0 \\ (10^8 K_d - 21.25)^2 &> 4 * 10^8 \\ 10^8 K_d - 21.25 &> 2 * 10^4 \\ K_d * 10^8 &>\approx 2 * 10^4 \\ K_d &> 2 * 10^{-4} \end{aligned}$$
>
> $\hookrightarrow$ thus, this criterion of K_d makes the 2nd order controller overdamped.

> To ensure that the system is stable, all of the coefficients in the 2nd order system characteristic polynomial equation must be positive.
>
> $$\begin{aligned} 10^8 K_d - 21.25 &> 0 \\ 10^8(1 + K_p) &> 0 \end{aligned}$$
>
> $$K_d > 2.125 * 10^{-7}, K_p >\approx 0$$
>
> $\hookrightarrow$ thus, with the coefficients all positive, the system is ensured to be stable.

An example of controller values are $K_p = 0$, $K_d = 2.5 * 10^{-4}$.
This yields a polynomial equation of:

$$s^2 + 24978.75s + 10^8$$

$\hookrightarrow$ $s \approx -19971.65, -5007.10$, both negative and makes a system stable. It also decays extremely fast. It is a very strong set of poles.

$$T_s \approx \frac{4}{5007} \approx 0.0008s \leq 0.05s$$

ece141_project_q2.m

C:\Users\wesle\OneDrive\Documents\MATLAB\ece141_project_q2.m

```
% PD controller
Kp = 0;
Kd = 2.5e-4;
Cpd = pid(Kp,0,Kd);

% Closed-loop transfer function (for reference)
T = feedback(Cpd*H,1);

disp('Closed-loop poles:')
pole(T)

disp('Step info:')
stepinfo(T)

figure;
step(T, 0.01);
grid on
title('Closed-loop step response with PD controller')
```

Command Window

```
>> ece141_project_q2
Closed-loop poles:

ans =

   1.0e+04 *

   -1.9972
   -0.5007

Step info:

ans =

  struct with fields:

        RiseTime: 0
   TransientTime: 9.3139e-04
    SettlingTime: NaN
     SettlingMin: 0
     SettlingMax: 0.7879
       Overshoot: Inf
      Undershoot: Inf
            Peak: 0.7879
        PeakTime: 9.2234e-05
```

Code Snippet 7: In this Matlab code snippet, the closed loop poles were calculated from the original controller constants K_p and K_d. The poles are equivalent to the values that were calculated earlier. The step info were also calculated in Matlab and analyzing the transient time, it is much less than the specified 0.005s maximum. The transient time is calculated to be 0.00093s.

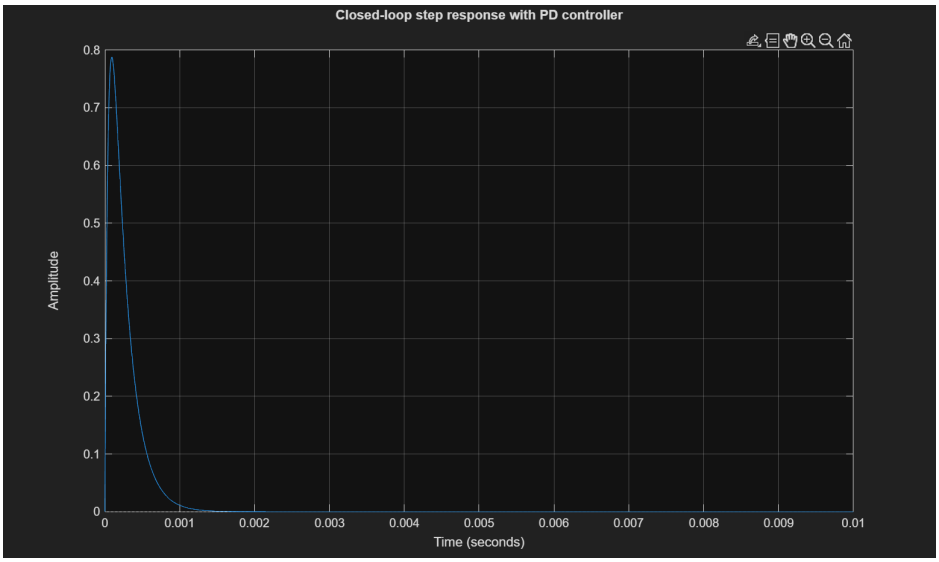

Figure 1: This figure represents the closed-loop step response with a PD controller with the given specifications above.
In this figure, it is shown that the system experiences a disturbance at time = 0 seconds.
Then, the controller brings it back down to 0 within the 0.005s specification requirement.

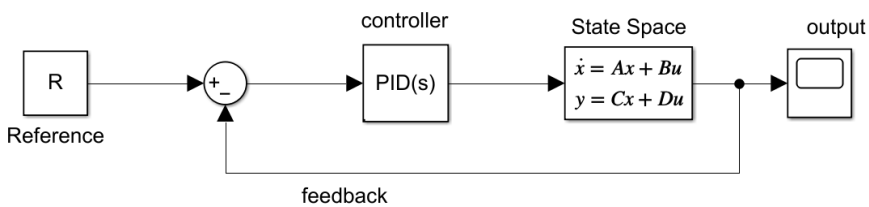

Figure 2: This figure represents the complete block diagram of the system. It highlights the reference input, the PID controller that we are designing, the state spaces, and the feedback and ultimately, the output. The state space block takes the previous values for the matrices of **A**, **B**, **C**, and **D**.

The system also uses the initial condition $\begin{bmatrix} 8 & -8 \end{bmatrix}$;

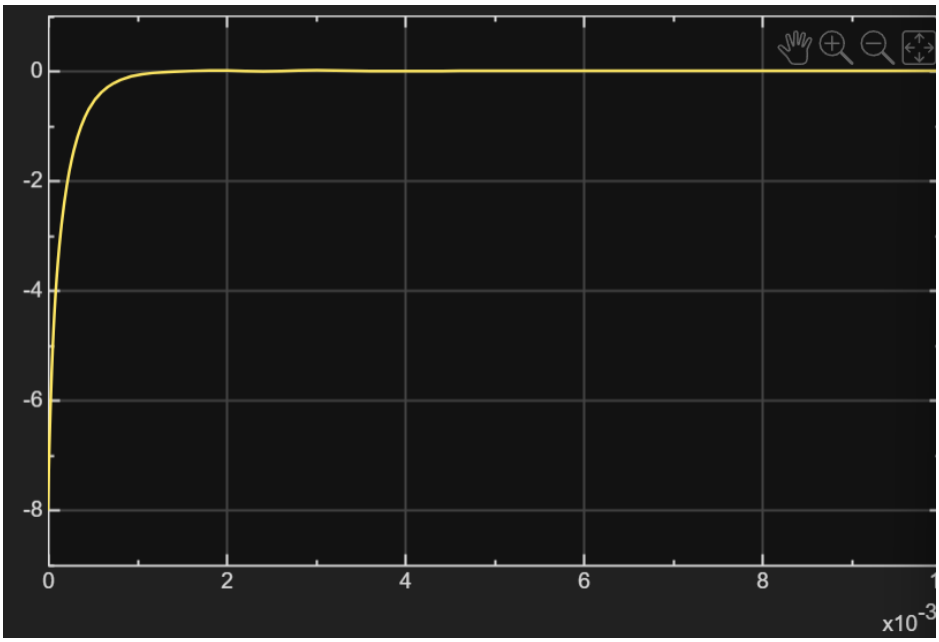

Figure 3: This figure is the graph of the output of the entire system due to a disturbance. The disturbance is modeled as the initial condition as $\begin{bmatrix} 8 & -8 \end{bmatrix}$. The PD controller uses the values:

$$K_p = 0, K_i = 0, K_d = 2.5 * 10^{-4}$$

designed so that the controller is overdamped, specifically to bring the output value back down to 0 as fast as possible with the least amount of oscillation. As seen in Figure 3, we see that the settling time is way below the specified settling time of 0.05s.

Summary – Part 6 The goal was to design a controller such that it would cause the unstable system's output to decay to 0 in 0.05 seconds or less and without overshoot. In my solution, the linearized system from Part 5 was implemented in Simulink using a State-Space block with output $y = \delta v_{dc}$ and initial condition $\delta x(0) = \begin{bmatrix} 8 & -8 \end{bmatrix}^T$. A stiff solver(ode23s) was used. Since the open-loop transfer function $H(s) = \dfrac{10^8}{s^2 - 21.25s + 10^8}$ is unstable, a proportional controller alone cannot stabilize the system because it does not change the sign of the s-term in the characteristic equation. Therefore, a PD controller was selected. With $K_p = 0, K_i = 0$, and $K_d = 2.5*10^{-4}$, the closed loop characteristic polynomial becomes $s^2+24978.75s+10^8$, which has two real negative poles. The corresponding response is stable, non-oscillatory, and settles to zero well within 0.05s, satisfying the design requirements.

7 Simulink Implementation

Implement the original nonlinear model in Simulink, assuming the initial states of $x(0) = x_0 + \delta x(0)$ and with the same parameters as Part 6. Keep in mind that the input v_s and the states v_{dc} needs to be non-negative at all times, with the additional safety requirement that they shall not exceed $160V, 20Aand200V$ respectively. *Hint:* you can use a saturation block in Simulink and set its lower and upper limits to the corresponding safety bounds. Apply the controller found in Part 6 by placing it in the closed-loop with the nonlinear model in Simulink. Since we are translating the system back to the original system, adjust the controller input appropriately, and the output's settling value. If needed, redesign your controller. Comment on the differences you observe as you implement the controller in the linearized vs the nonlinear model.

Goal: To implement the original nonlinear model in Simulink considering the safety requirements. Compare the linearized model with the nonlinearized model.

Linearized Model:

$$\delta\dot{x} = A\delta x + B\delta u$$

$$y = \delta x_2 = \delta v_{dc}$$

$\hookrightarrow$ Then, close the loop around a nonlinear model.

> Using the values restated in Part 6, the equilibrium state is
>
> $$x_0 = \begin{bmatrix} i_{L0} \\ v_{dc0} \end{bmatrix} = \begin{bmatrix} 0.33 \\ 80 \end{bmatrix}$$

> Calculating for the initial condition:
>
> $$x(0) = x_0 + \delta x(0)$$
>
> $$\delta x(0) = \begin{bmatrix} 8 \\ -8 \end{bmatrix}$$
>
> $$x(0) = \begin{bmatrix} 0.33 \\ 80 \end{bmatrix} + \begin{bmatrix} 8 \\ -8 \end{bmatrix} = \begin{bmatrix} 8.33 \\ 72 \end{bmatrix}$$
>
> thus, the nonlinear system has an initial condition of:
> $\hookrightarrow i_L(0) = 8.33A$
> $\hookrightarrow v_{dc}(0) = 72V$

Linearization

$$\begin{aligned} \delta v_s &= v_s - v_{s0} \\ v_s &= v_{s0} + \delta v_s \\ v_s &= 80 + \delta v_s \\ &\Downarrow \\ \delta v_{dc} &= v_{dc} - 80 \end{aligned}$$

Goal: drive $v_{dc} \to 80V$, driving $\delta v_{dc} \to 0$ (equilibrium).

In the controller,
$\hookrightarrow$ controller sees $v_{dc} - 80$
$\hookrightarrow$ controller outputs δv_s
$\hookrightarrow$ actual input to plant is $80 + \delta v_s$

Nonlinear Dynamics

Inductor current:

$$\dot{i_L} = \frac{1}{L}(v_s - v_{dc})$$

$$\dot{i_L} = 10000(v_s - v_{dc})$$

Capacitor voltage:

$$\dot{v_{dc}} = \frac{1}{C}(i_L - \frac{v_{dc}}{R} - \frac{P}{v_{dc}})$$

$$\dot{v_{dc}} = 10000(i_L - \frac{v_{dc}}{1000} - \frac{20}{v_{dc}})$$

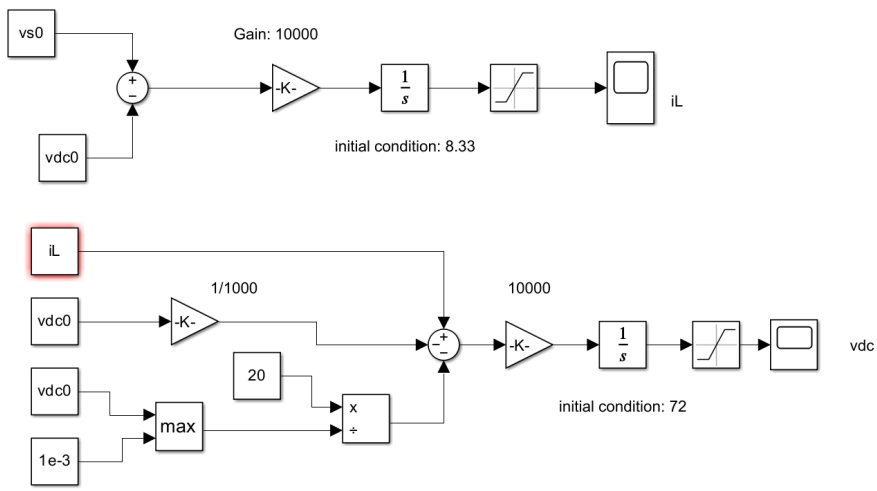

Figure 4: The closed-loop Simulink model was built by placing the Part 6 Derivative-only controller around the nonlinear plant. Since the controller was designed using deviation variables, the feedback error for the nonlinear system was defined as $e(t) = 80 - v_{dc}(t)$, so that regulating the nonlinear output to $80V$ is equivalent to driving $\delta v_{dc} \to 0$. The controller output is δv_s, and the acutal plant input is formed using $v_s = 80 + \delta v_s$. A saturation block was added to enforce the source voltage safety bound as listed in the specification. Inside the plant subsystem, the nonlinear dynamics $\dot{i_L} = 10000(v_s - v_{dc})$ and $\dot{v_{dc}} = 10000(i_L - \frac{v_{dc}}{1000} - \frac{20}{v_{dc}})$ were implemented using gains, sums, divide, and integrator blocks. The state variables were also constrained to the required safety ranges $0 \leq i_L \leq 20A$ and $0 \leq v_{dc} \leq 200V$.

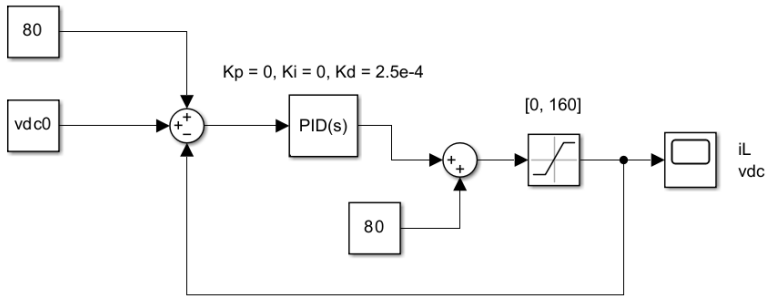

Figure 5: Closed loop block diagram in Simulink of the nonlinear system with the Part 6 PD controller. The capacitor voltage v_dc is fed back and compared with the reference value of 80V to form the error $e(t) = 80 - v_{dc}(t)$. The controller generates the deviation input δv_s. A saturation block enforces the physical bounds $0 \leq v_s \leq 160V$. The nonlinear plant subsystem implements the original dynamics and outputs the inductor current and the capacitor voltage: i_L and v_{dc} respectively.

Summary (Part 7)

The original nonlinear system was implemented in Simulink using the plant equations for the inductor current and capacitor voltage, which include the constant power load term P/v_{dc}. The controller designed in Part 6 was originally formulated using deviation variables, so the nonlinear implementation required translating the controller back to the original variables. The feedback error was defined as $e(t) = 80 - v_{dc}(t)$, ensuring that regulating the nonlinear output to 80V corresponds to driving the output δv_{dc} to zero. The controller output produces δv_s, and the actual plant input is obtained using $v_s = 80 + \delta v_s$.

I added saturation blocks to enforce the physical safety limits listed in the specifications in the project, ensuring that $0 \leq v_s \leq 160V$, $0 \leq i_L \leq 20A$, and $0 \leq v_{dc} \leq 200V$. Because the nonlinear model contains the term $\dfrac{P}{v_{dc}}$, a small lower bound was used in simulation to prevent division by zero. The system was simulated using the stiff solver `ode23s` to accurately capture the fast electrical dynamics.

The nonlinear closed-loop system successfully regulates the capacitor voltage to the desired equilibrium value of 80V while keeping all signals within their required limits. The nonlinear response shows small differences in the transient behavior due to the true nonlinear nature of the constant power load. The controller I designed remains effective at stabilizing the system and restoring the system to its operating point, or to its equilibrium.